

EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs

Maria Szlasa

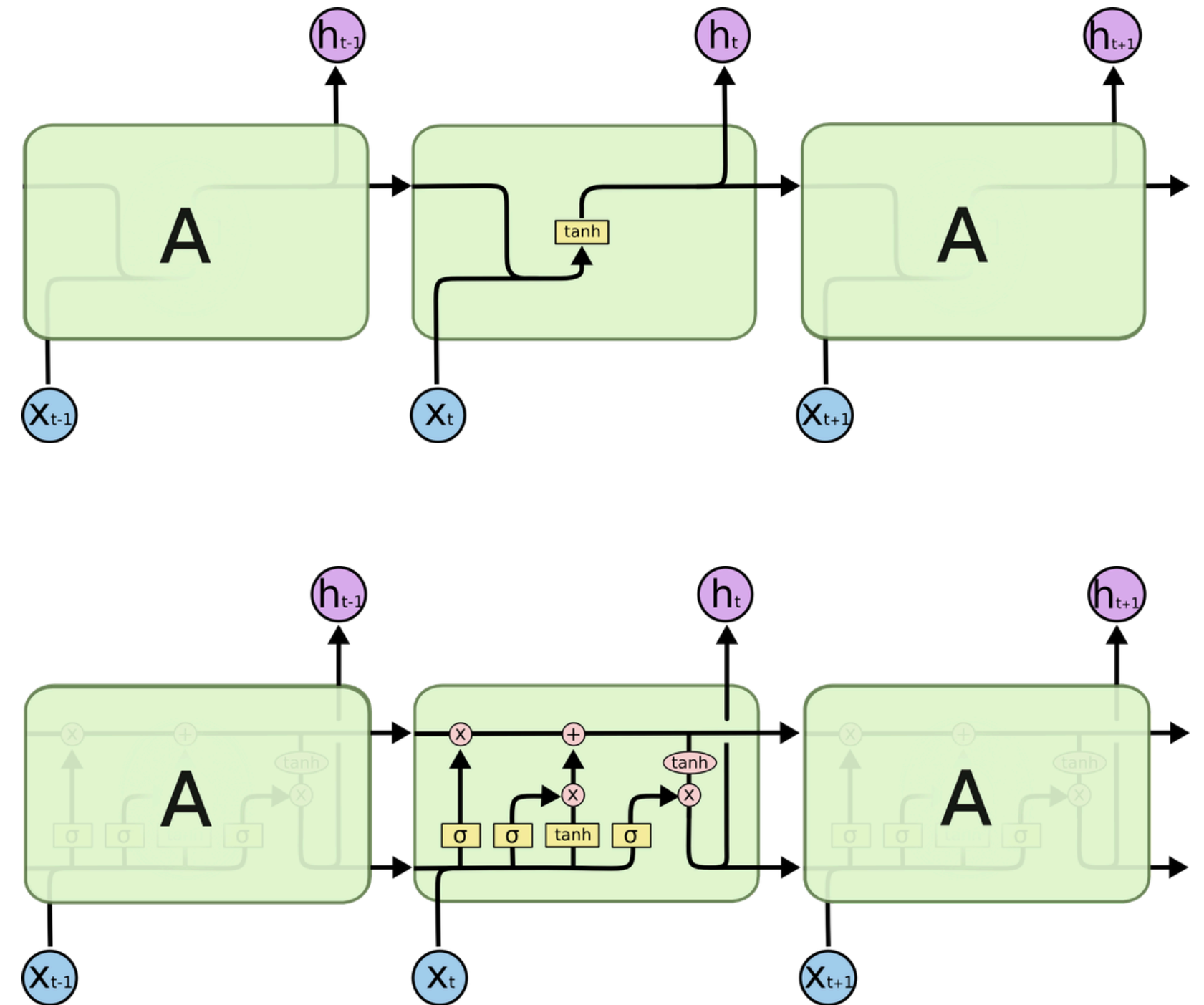
Agenda

1. Recap
 - a. LSTM
 - b. GRU
2. EvolveGCN

What is an LSTM?

A standard RNN repeating module is extremely simple, just a single tanh layer. This simplicity is its weakness: the network has no mechanism to selectively remember or forget information over long sequences.

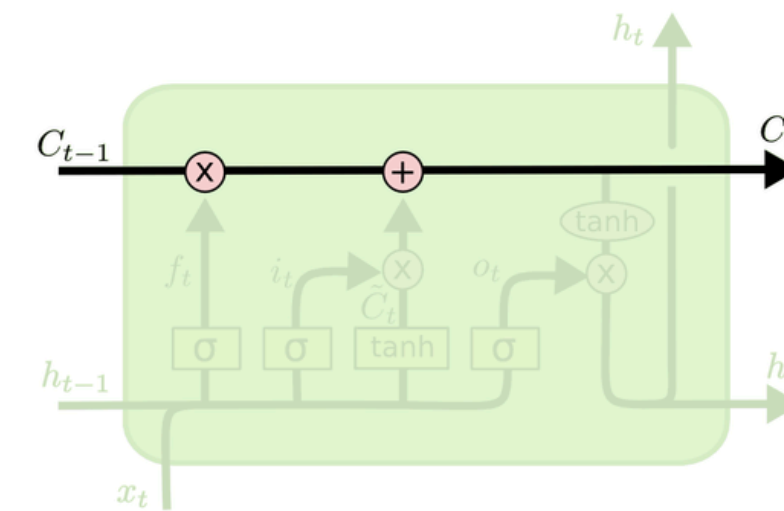
An LSTM replaces that single layer with four interacting layers. This extra complexity is precisely what allows LSTMs to decide what to remember, what to forget, and what to pass forward – making them far more powerful for sequential data.



The Core Idea: Cell State

The key innovation of LSTMs is the cell state, the horizontal line running through the top of the diagram. Think of it as a conveyor belt: information flows along it largely unchanged, with only minor, controlled modifications at each step.

This is what standard RNNs lack. Gradients can flow through the cell state over many timesteps without vanishing solving the long-term dependency problem.

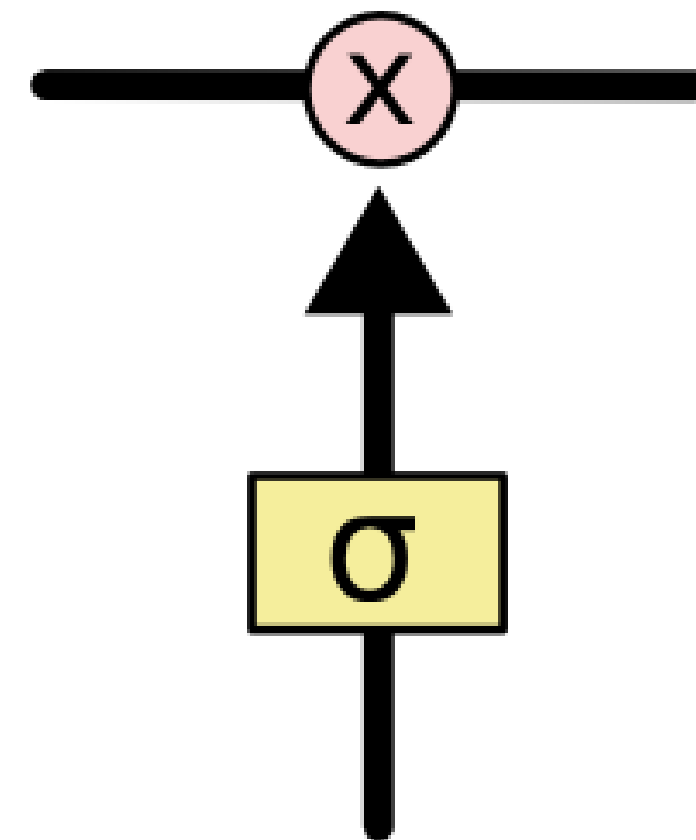


Gates: The Control Mechanism

The cell state is controlled by gates — structures that decide how much information to let through. Each gate is a sigmoid layer followed by pointwise multiplication.

The sigmoid outputs values between 0 and 1: 0 means "block everything", 1 means "let everything through". By learning these gate values, the LSTM learns when to remember and when to forget.

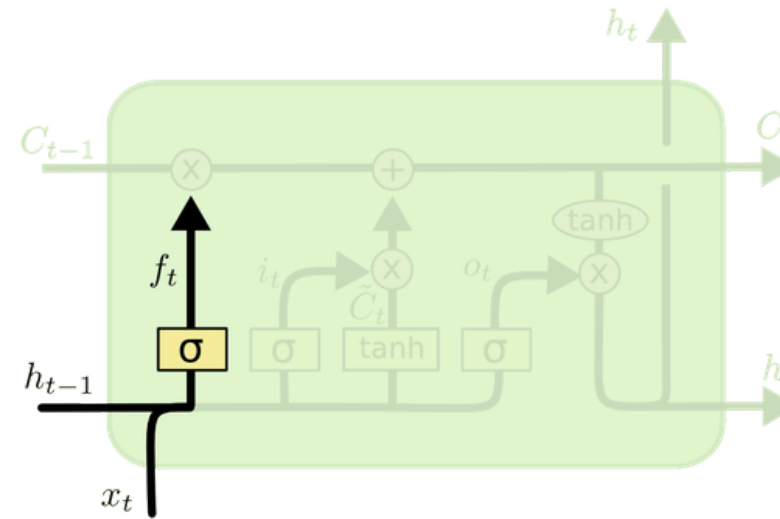
An LSTM has three such gates. Together, they give the network fine-grained control over its memory which is exactly why it can model long-range dynamics like the evolution of a graph over dozens of timesteps.



Step 1: Forget Gate

The first step asks: what should we throw away?

The forget gate looks at the previous hidden state h_{t-1} and the current input x_t , and outputs a number between 0 and 1 for each element of the cell state. Values close to 0 mean "forget this", values close to 1 mean "keep this."



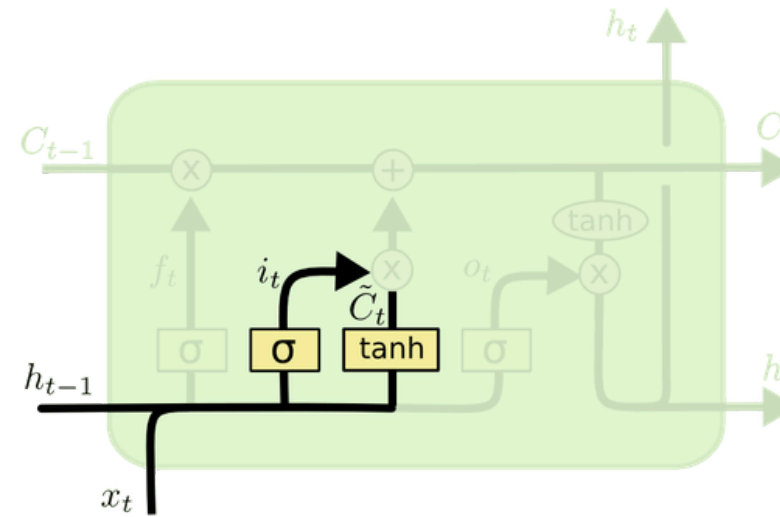
$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Step 2: Input Gate

The second step asks: what new information should we store?

This involves two operations happening in parallel. The input gate (sigmoid) decides which values to update. A tanh layer creates a vector of candidate values $\tilde{C}_t$, new information that could be written into memory.

Neither alone is sufficient: the input gate acts as a filter on the candidates, ensuring only relevant new patterns are written into the cell state.



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

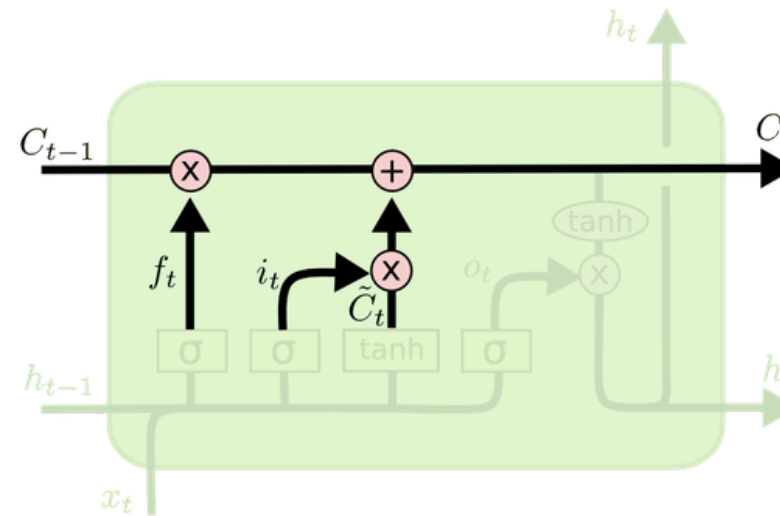
Step 3: Cell State Update

Now we actually update the cell state by combining the decisions from the previous two steps:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

We multiply the old state by the forget gate f_t erasing what we decided to forget. Then we add the filtered new candidates $i_t \odot \tilde{C}_t$ writing in what we decided to remember.

This single equation is why LSTMs work: the cell state is updated additively, which keeps gradients healthy during backpropagation through time.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

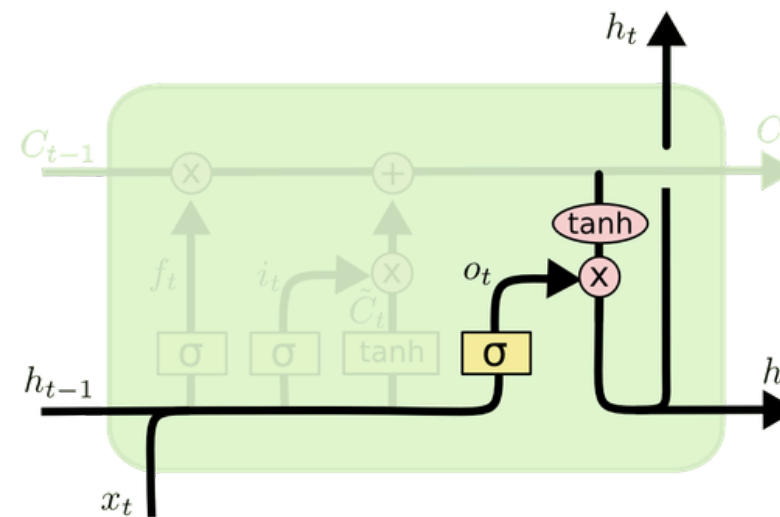
Step 4: Output Gate

The final step asks: what do we actually output?

The output gate (sigmoid) decides which parts of the cell state are relevant right now. The cell state is then pushed through tanh to rescale values to $[-1,1]$, and multiplied by the output gate.

$$h_t = o_t \odot \tanh(C_t)$$

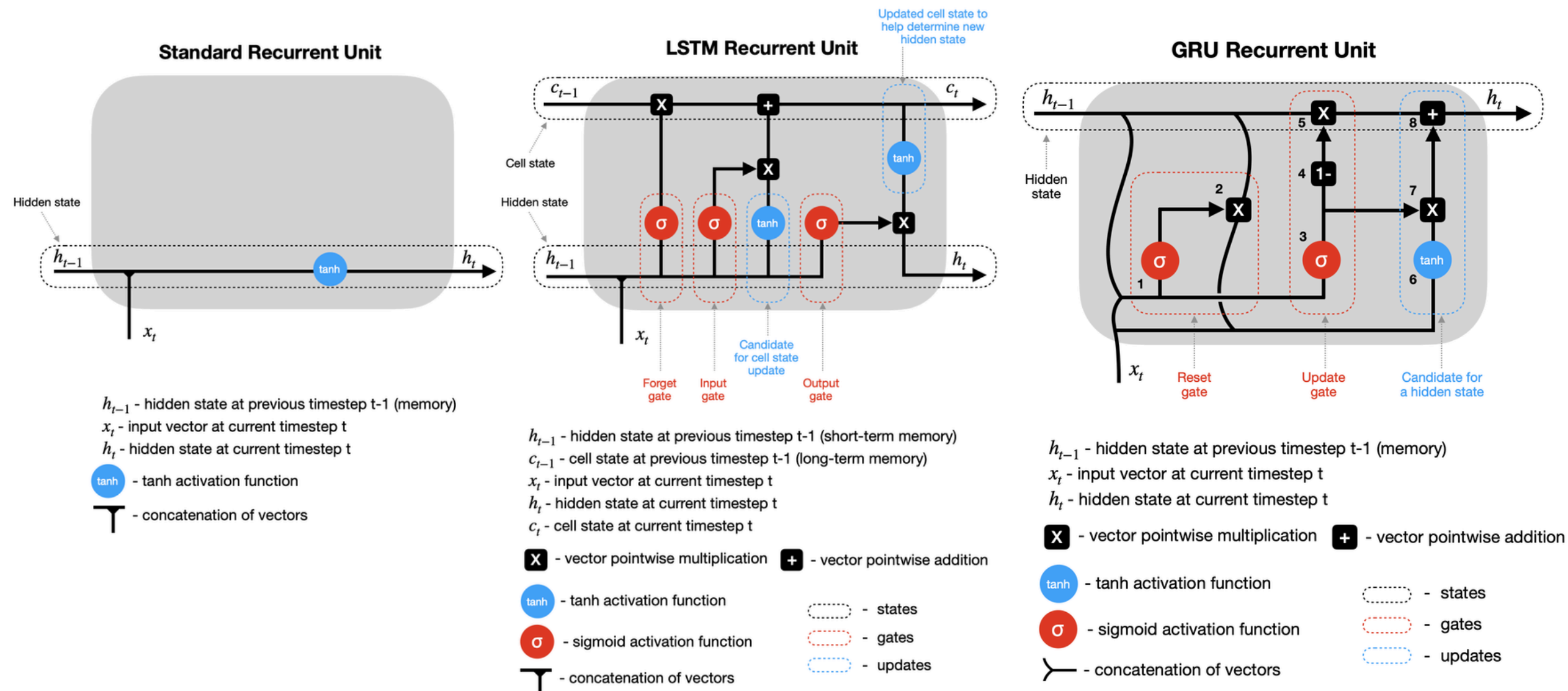
Crucially, h_t is not the full cell state it's a filtered, task-relevant summary of it. In EvolveGCN-O, this output becomes the next input. The updated weight matrix $W_t(l)$ is passed as the next timestep's input, closing the loop.



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$

GRU vs. LSTM: Simplifying the Architecture

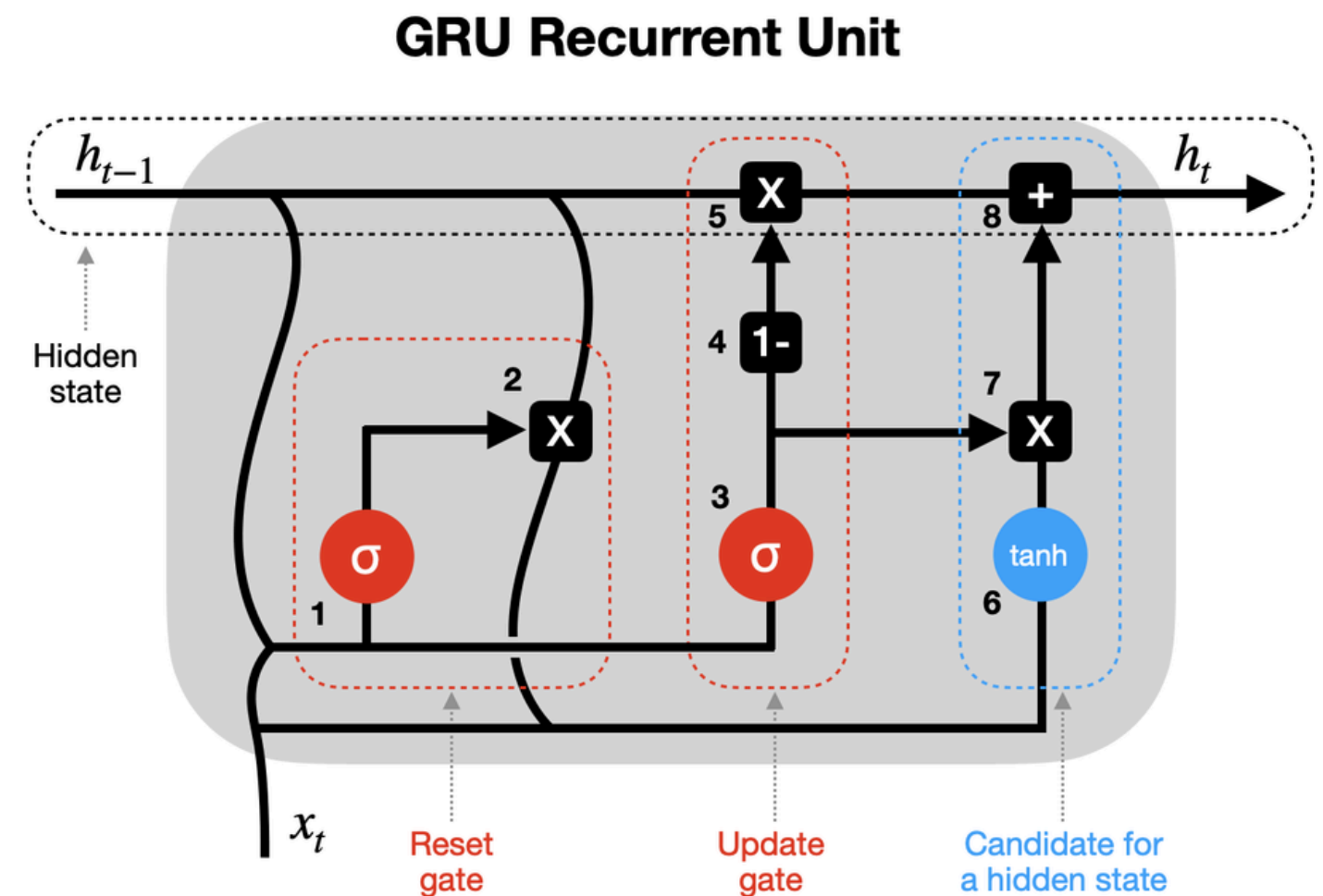
Before understanding GRU, it helps to see what it's replacing. A standard RNN has a single hidden state. Simple, but prone to vanishing gradients over long sequences. LSTM solves this by introducing a separate cell state as long-term memory, plus three gates to control information flow. The result is powerful, but complex. LSTM has four interacting layers and two separate memory channels (hidden state + cell state), which means more parameters to train and more computation at every timestep.



How GRU Works: Two Gates, One Memory

GRU achieves what LSTM does using only two gates and no separate cell state, just a single hidden state carries memory between timesteps.

Reset gate (steps 1–2): looks at the previous hidden state h_{t-1} and current input x_t , and outputs values between 0 and 1. Multiplying by h_{t-1} controls how much of the past feeds into the new candidate, a value of 0 effectively resets the memory.



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

$\times$ - vector pointwise multiplication $+$ - vector pointwise addition

$\tanh$ - tanh activation function

σ - sigmoid activation function

Y - concatenation of vectors

dashed black box - states

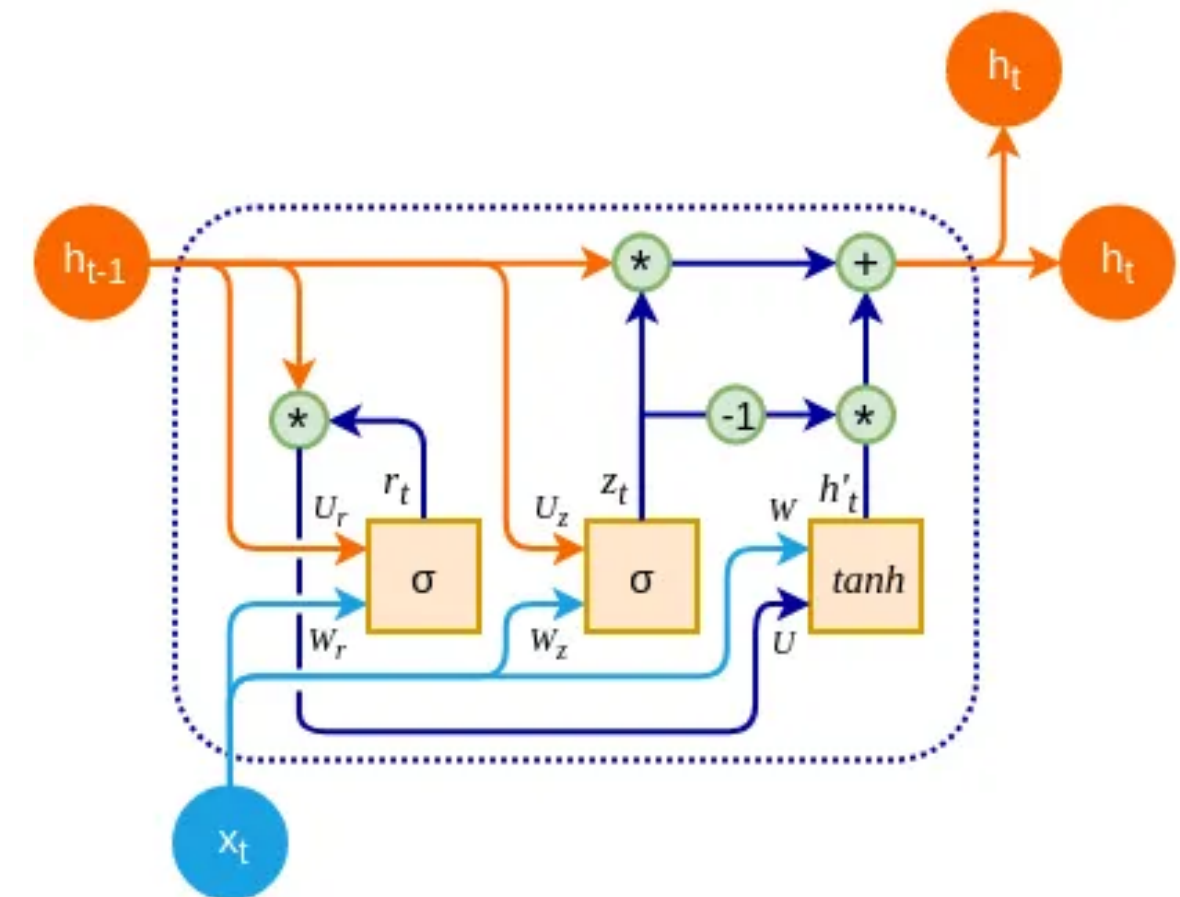
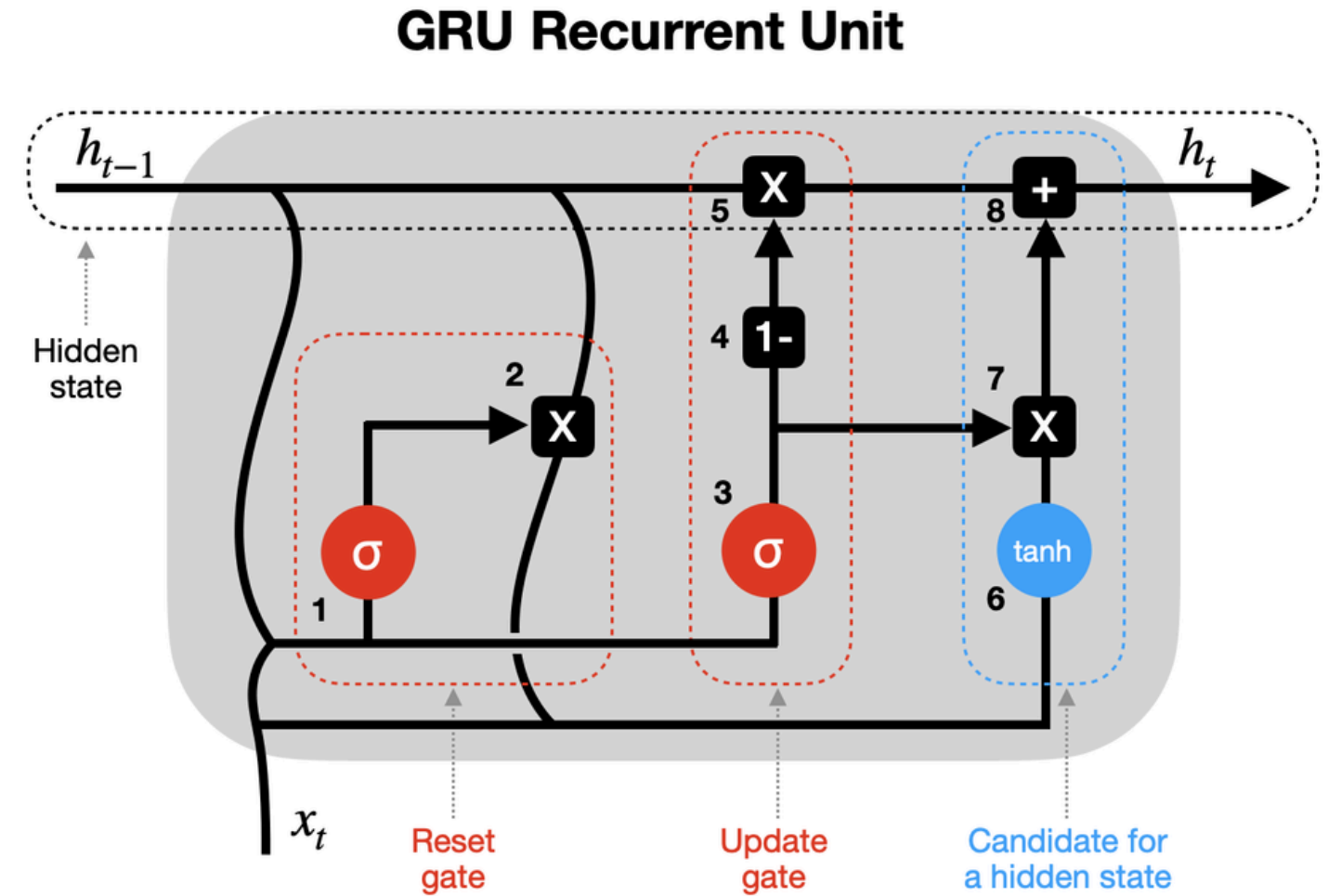
dashed red box - gates

dashed blue box - updates

How GRU Works: Two Gates, One Memory

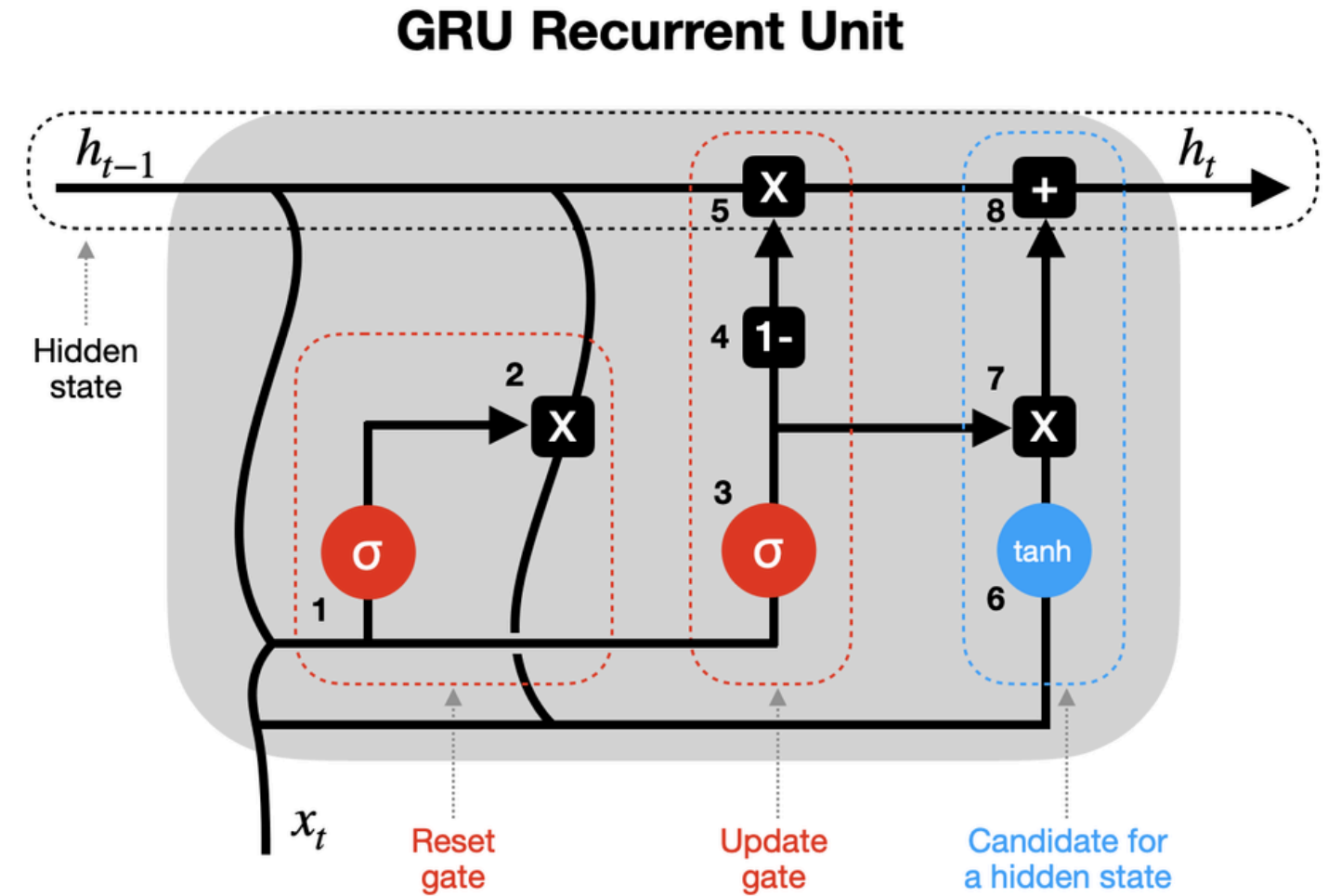
GRU achieves what LSTM does using only two gates and no separate cell state, just a single hidden state carries memory between timesteps.

Reset gate (steps 1–2): looks at the previous hidden state h_{t-1} and current input x_t , and outputs values between 0 and 1. Multiplying by h_{t-1} controls how much of the past feeds into the new candidate, a value of 0 effectively resets the memory.



How GRU Works: Two Gates, One Memory

Update gate (steps 3–5): decides how much of the old hidden state to keep versus replace. It subtracts its output from 1 and multiplies by h_{t-1} , so a high update gate value means "keep the past as-is," while a low value opens the door to new information.



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

$\times$ - vector pointwise multiplication $+$ - vector pointwise addition

$\tanh$ - tanh activation function

σ - sigmoid activation function

$\cup$ - concatenation of vectors

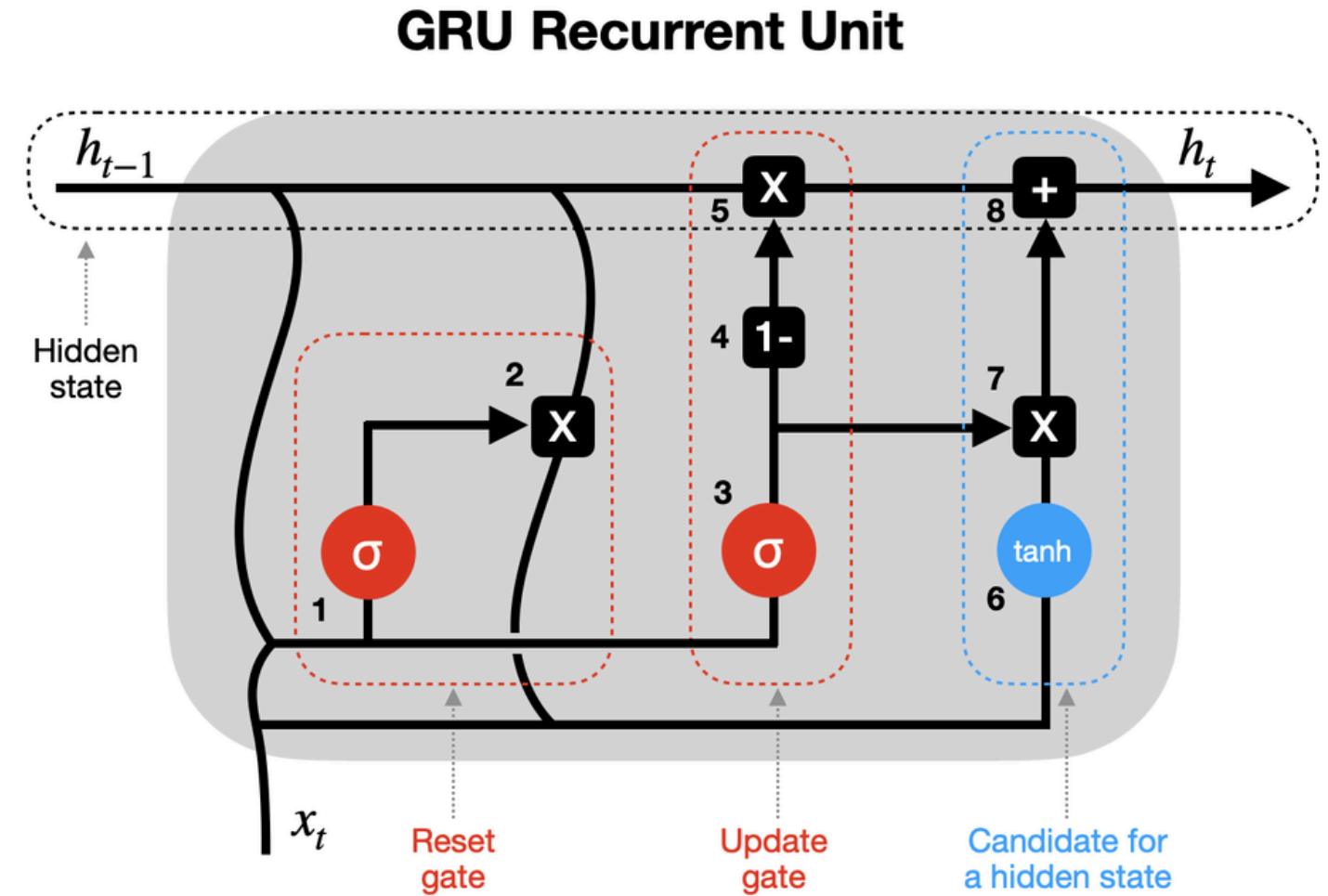
- states

- gates

- updates

How GRU Works: Two Gates, One Memory

Hidden state candidate (steps 6–8): combines the reset hidden state with new input through tanh, then blends it with the old hidden state using the update gate. The result is h_t , the new memory passed forward.



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

X - vector pointwise multiplication **+** - vector pointwise addition

tanh - tanh activation function

σ - sigmoid activation function

Y - concatenation of vectors

— - states

- - - - gates

- - - - updates

From Static to Dynamic: The Problem

Graph neural networks are designed for a fixed, static graph – one adjacency matrix A , one feature matrix X , one forward pass. In practice, graphs are rarely static.

In a Bitcoin transaction network, new transactions appear every second and the nature of accounts changes over time – a wallet that behaved legitimately yesterday may be involved in money laundering today. In a social network, friendships form and dissolve, and the influence of a user shifts as their connections change. In a citation network, the importance of a paper evolves as new work builds on it or supersedes it.

In all these cases, the graph at time $t+1$ is meaningfully different from the graph at time t . A model trained on a static snapshot will not capture these dynamics. We need an architecture that processes a sequence of graphs and learns how that sequence evolves.

The Flaw in the Obvious Approach

In prior GNN+RNN methods, the pipeline is:

$$\text{Graph}(t) \rightarrow \text{GCN} \rightarrow h_t \rightarrow \text{RNN} \rightarrow \text{temporal dynamics}$$

A single shared GCN extracts node embeddings at each timestep. The RNN then models how those embeddings change over time.

The implicit assumption is that every node has an embedding at every timestep – the RNN needs a complete history per node. This assumption fails in three practical scenarios:

- a new node appears after training – the RNN has no history for it
- nodes appear and disappear irregularly – the RNN cannot learn these patterns reliably
- in the extreme case, the node sets at t and $t+1$ are completely disjoint

The Key Insight: Evolve the Model, Not the Embeddings

EvolveGCN proposes the following reformulation. Instead of using the RNN to evolve node embeddings, use the RNN to evolve the GCN weight matrices:

$$\begin{aligned}W_t &= \text{RNN}(W_{t-1}) \\H_t &= \text{GCN}(A_t, X_t; W_t)\end{aligned}$$

The weight matrices carry the temporal information. At each timestep, the RNN produces an updated GCN, one whose parameters reflect the history of the graph sequence up to that point. Node embeddings are computed fresh from the current graph using those evolved weights.

This resolves the node identity problem: a node that was not present at previous timesteps is simply processed by the current evolved GCN with no need for historical node-level information.

An important consequence for model complexity: the GCN weights are not free parameters. They are outputs of the RNN. Only the RNN parameters are trained via backpropagation. Model size therefore does not grow with the number of timesteps.

Building Block: The GCN Layer

We first define precisely what a single GCN layer computes. At timestep t , layer l applies:

$$H_t^{l+1} = \text{GCONV}(A_t, H_t^l, W_t^l) = \sigma(A_{\text{hat}}_t * H_t^l * W_t^l)$$

where $H_{\{t\}}(0)=X_{\{t\}}$ is the matrix of input node features, $W_{\{t\}}(l)$ is the weight matrix at layer l and time t , and A^t is the symmetrically normalized adjacency matrix:

$$A_{\text{hat}} = D_{\text{tilde}}^{-1/2} * A_{\text{tilde}} * D_{\text{tilde}}^{-1/2}$$

$$A_{\text{tilde}} = A + I,$$

$$D_{\text{tilde}}_{ii} = \sum_j A_{\text{tilde}}_{ij} \quad (\text{stopień wężła i po dodaniu self-loop})$$

Adding the identity matrix I introduces self-loops, so each node aggregates its own features alongside its neighbors. The symmetric normalization by $D_{\text{tilde}}^{-1/2}$ rescales each edge (i,j) by $1/d_i \cdot d_j$, preventing high-degree nodes from disproportionately influencing the aggregation.

The operation $A^t H_t(l) W_t(l)$ aggregates neighborhood information and applies a linear transformation. σ is ReLU for hidden layers and softmax or identity for the output layer.

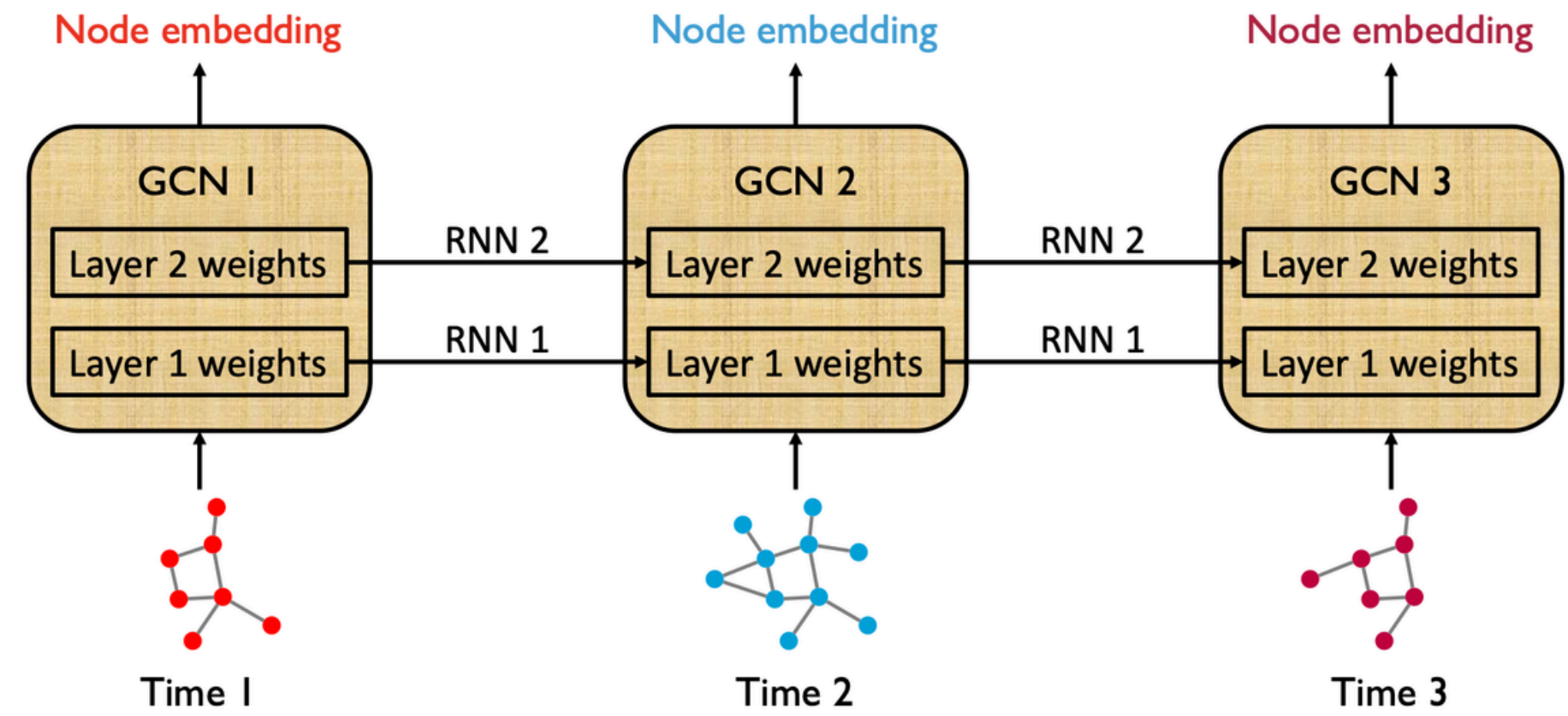
The Full Architecture: EvolveGCN

Each column is one timestep, each containing its own GCN with distinct weight matrices, not a shared model.

Information flows in two directions: vertically through GCN layers within a single timestep, and horizontally through the RNN across timesteps.

There is a separate RNN per layer, since weights at different depths capture different levels of structural abstraction.

The paper proposes two concrete instantiations of this architecture, differing in how the GCN weights are treated inside the RNN.

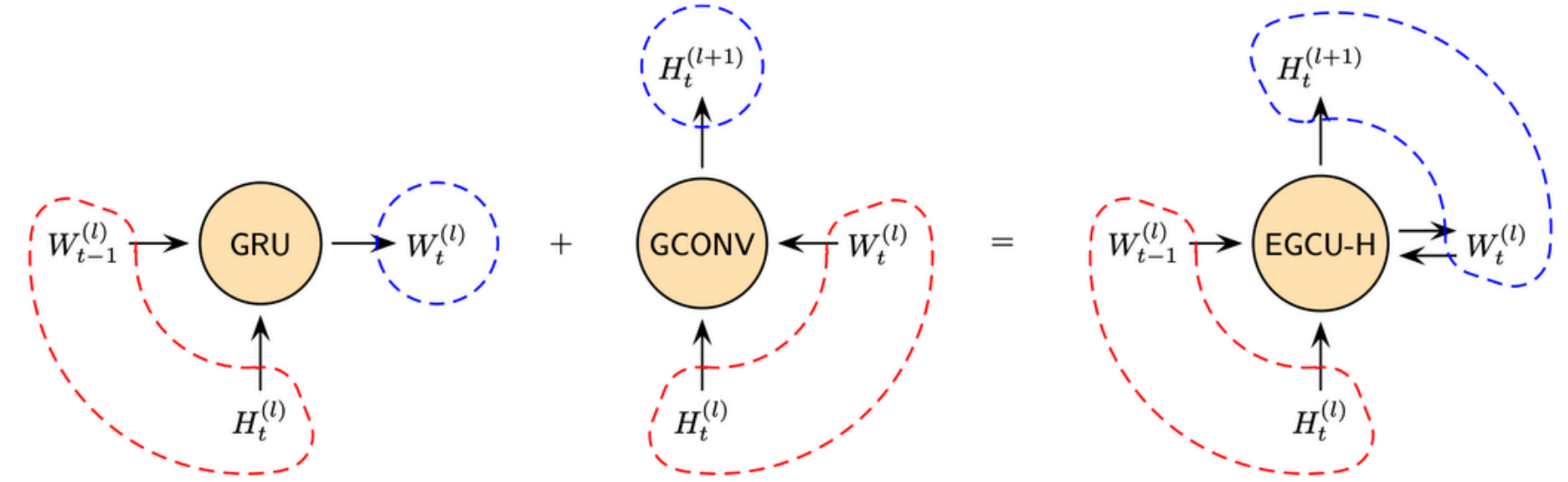


Two Versions: **-H** and **-O**

EvolveGCN-H treats the GCN weight matrix as the hidden state of a GRU:

$$W_t^{(l)} = \text{GRU}(H_t^{(l)}, W_{t-1}^{(l)})$$

The node embedding matrix $H_t^{(l)}$ serves as the GRU input. The previous weight matrix $W_{t-1}^{(l)}$ is the hidden state. The GRU update produces new weights $W_t^{(l)}$. This version incorporates the current state of the nodes into the weight update.



(a) EvolveGCN-H, where the GCN parameters are hidden states of a recurrent architecture that takes node embeddings as input.

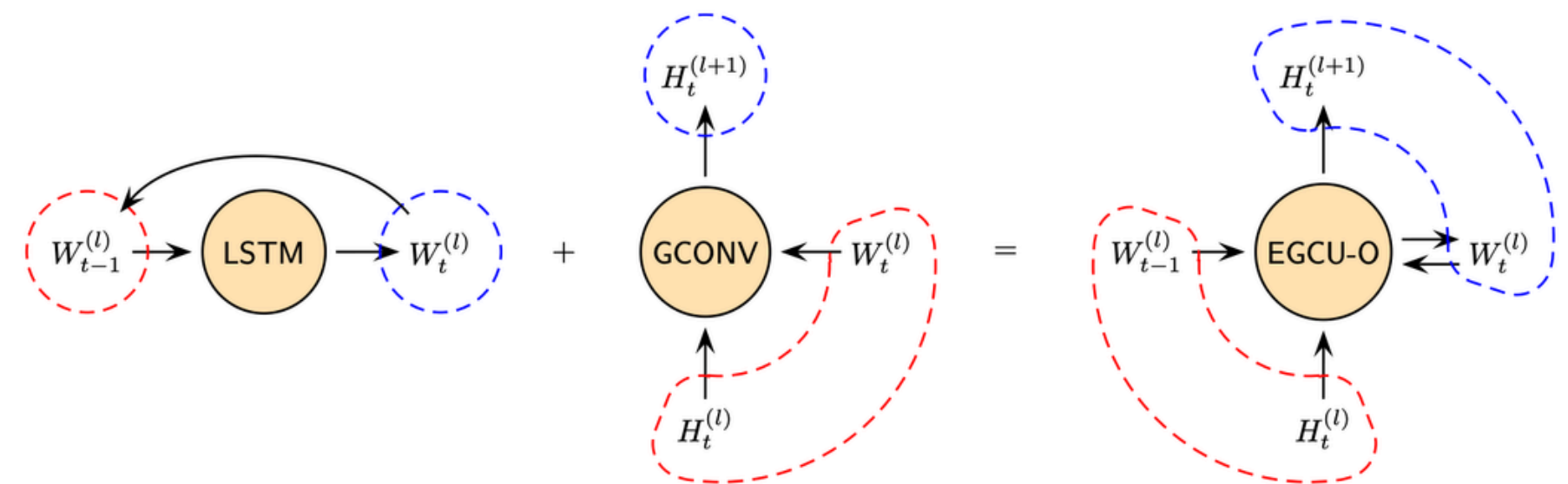
Two Versions: -H and -O

EvolveGCN-O treats the GCN weight matrix as the input and output of an LSTM:

$$W_{\{t\}}^{(l)} = \text{LSTM}(W_{\{t-1\}}^{(l)})$$

Node embeddings are not used. The LSTM maintains an internal cell state as implicit memory and maps previous weights to new weights. This version focuses exclusively on how the weight matrices themselves evolve.

In both cases the procedure inside one unit is the same: first produce updated weights via the recurrent architecture, then apply graph convolution using those weights.



(b) EvolveGCN-O, where the GCN parameters are input/outputs of a recurrent architecture.

EGCU: Definition

Combining the recurrent weight update with graph convolution gives the Evolving Graph Convolution Unit (EGCU).

Step 1 (-H): $W_t^l = \text{GRU}(H_t^l, W_{t-1}^l)$

Step 1 (-O): $W_t^l = \text{LSTM}(W_{t-1}^l)$

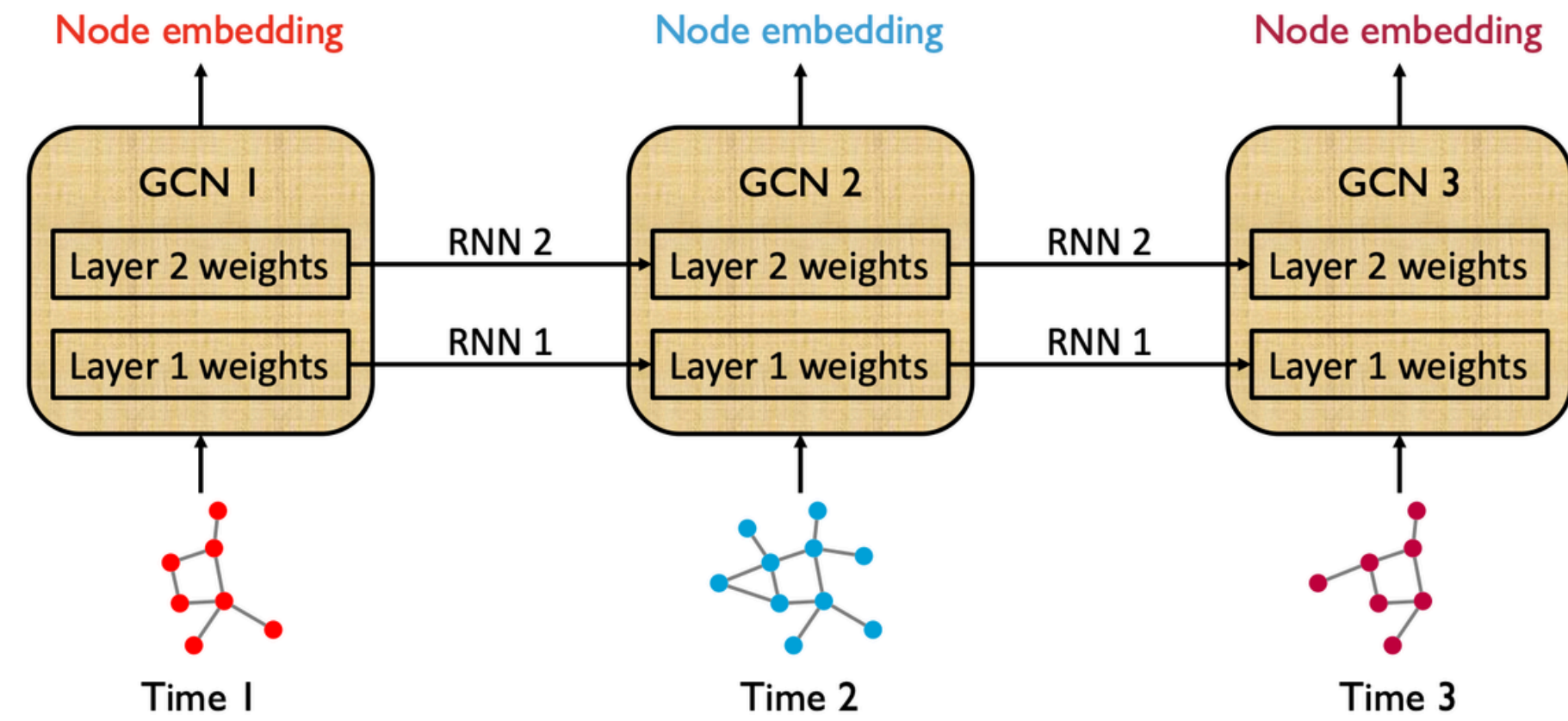
Step 2 (both): $H_t^{l+1} = \text{GCONV}(A_t, H_t^l, W_t^l)$

Step 1: the GRU/LSTM produces updated weights from the current node embeddings and the previous weights.

Step 2: those weights are immediately used to perform graph convolution on the current graph.

EGCU: How Units Compose

Stacking EGCU units vertically gives a multi-layer GCN for one timestep. Unrolling horizontally over T timesteps gives the full model. The resulting computation forms a lattice: $H_t(l)$ flows upward through layers, $W_t(l)$ flows rightward through time. At every node of this lattice, one EGCU unit takes the previous weights and current embeddings as input, and produces updated weights and new embeddings as output.



When to Use -H vs -O

The choice between the two versions depends on the relative informativeness of node features versus graph structure.

When node features carry meaningful signal, rich attributes, precomputed feature vectors. The H version can exploit this by conditioning the weight update on the current node embeddings. When node features are weak or uninformative (e.g., one-hot degree encoding), the O version is preferable as it focuses entirely on the structural evolution captured in the weight sequence.

	EvolveGCN-H	EvolveGCN-O
RNN type	GRU	LSTM
Uses node embeddings	Yes	No
GCN weights role	Hidden state	Input / Output
Recommended when	Node features are informative	Graph structure drives dynamics

Implementation: From Vectors to Matrices

Standard GRU and LSTM are defined for vector inputs and vector hidden states. Here the hidden state is a weight matrix $W(l) \in \mathbb{R}^{d \times k}$.

The solution is straightforward: apply the GRU column-wise across $W(l)$. The update equations remain structurally identical:

$$Z_t = \text{sigma}(W_Z X_t + U_Z H_{\{t-1\}} + B_Z)$$

$$R_t = \text{sigma}(W_R X_t + U_R H_{\{t-1\}} + B_R)$$

$$H_{\text{tilde}} = \tanh(W_H X_t + U_H (R_t \odot H_{\{t-1\}}) + B_H)$$

$$H_t = (1 - Z_t) \odot H_{\{t-1\}} + Z_t \odot H_{\text{tilde}}$$

All variables are now matrices; $\odot$ denotes elementwise multiplication.

Implementation: Dimension Mismatch

In the -H version, the GRU requires input and hidden state to have the same number of columns k . The node embedding matrix $H_t(l)$ has n rows, one per node, where n is variable. A summarize function reduces it to exactly k representative rows:

$$\begin{aligned} y_t &= H_t^l * p / \|p\| \\ i_t &= \text{top-}k\text{-indices}(y_t) \\ Z_t &= [H_t^l \odot \tanh(y_t)]_{i_t} \end{aligned}$$

p is a learned parameter vector, independent of t , separate per layer. The selection is differentiable. Gradients flow through the selected rows. The full -H update becomes:

$$W_t^l = g(\text{summarize}(H_t^l, k)^T, W_{\{t-1\}^l})$$

Summary

EvolveGCN introduces a single architectural change relative to prior GNN+RNN methods: temporal information is stored in the GCN weight matrices rather than in node embeddings. An RNN – GRU in the -H version, LSTM in the -O version, produces updated weights at each timestep.

This removes the dependency on node identity across timesteps. Nodes can appear, disappear, or be entirely new at test time without affecting the model's ability to process them.

Two variants provide flexibility depending on whether node features or graph structure are the primary source of signal. Only the RNN parameters are trained; model size is independent of sequence length.

Experimental results show generally stronger performance on supervised tasks compared to related methods, with the caveat that unsupervised autoencoder baselines remain competitive for link prediction, and that structural regime changes not seen during training degrade all methods regardless of architecture.

<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

<https://towardsdatascience.com/gru-recurrent-neural-networks-a-smart-way-to-predict-sequences-in-python-80864e4fe9f6/>

<https://arxiv.org/pdf/1902.10191>